

Binary Trees and Tree Data Structures

Course index and how to use this material

Who this is for

Someone who has never studied trees. You should be comfortable with C++ basics: functions, pointers at a beginner level, vector, queue, stack, and simple recursion such as factorial. Nothing else is assumed. Recursion is reintroduced from scratch in Part 1 where it is needed.

What you will be able to do at the end

Open any LeetCode Easy tree problem and solve it in ten minutes. Open a typical Medium tree problem and know within a minute which of five patterns it belongs to, then implement it. More importantly, look at a problem you have never seen and ask the right questions:

What information does each subtree need to return?

Is this DFS, BFS, or bottom-up?

What should my base case be?

What do I combine from the left and the right?

Memorising twenty solutions is not the goal, and does not work. Recognising five patterns does.

The parts

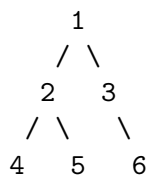
Part	Topic	What it gives you
1	Tree Fundamentals	vocabulary, TreeNode, why recursion fits trees
2	Recursive DFS	preorder, inorder, postorder, and when each is used
3	BFS and Level Order	the queue template and all its variations
4	Iterative Traversals	replacing the call stack with your own stack
5	Basic Tree Properties	your first ten recursive functions

Part	Topic	What it gives you
6	Bottom-Up Thinking	returning information upward, avoiding $O(N^2)$
7	Diameter	the classic bottom-up problem
8	Balanced Binary Tree	the sentinel value pattern
9	Path Problems	root to leaf, backtracking, maximum path sum
10	Core LeetCode Patterns	invert, subtree, LCA, construction, serialization, BST
11	Tree Views	horizontal distance, top, bottom, vertical, boundary
12	Morris Traversal	inorder in $O(1)$ extra space
13	How to Think About Tree Problems	the pattern recognition checklist
14	Practice Problem Sets	graded problems with hints, no solutions
15	Revision Sheet	every template on a few pages

Read them in order. Parts 5, 6 and 7 are the spine of the whole course. If you are short of time, never skip those.

The tree used throughout

Every part uses the same example tree so that you build familiarity with one shape instead of re-reading a new diagram every page.



```

preorder    : 1 2 4 5 3 6
inorder     : 4 2 5 1 3 6
postorder   : 4 5 2 6 3 1
level order : 1 | 2 3 | 4 5 6
height      : 3 nodes, 2 edges
diameter    : 4 edges (4 -> 2 -> 1 -> 3 -> 6)
top view    : 4 2 1 3 6
bottom view : 4 2 5 3 6
right view  : 1 3 6
boundary    : 1 2 4 5 6 3

```

By the end of Part 11 you should be able to produce that whole block from the picture, from memory, in under two minutes. That is a good self-test.

Keep this in your practice file:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

TreeNode* buildSampleTree() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->right = new TreeNode(6);
    return root;
}
```

Run every algorithm you learn on this tree locally. Reading code is not learning code.

How each part is organised

Every major topic follows the same shape:

1. what the thing means, in plain words
2. a picture
3. the algorithm in pseudocode
4. C++ implementation, LeetCode style
5. a dry run on a small tree
6. time and space complexity, with the reason
7. common mistakes
8. Check Yourself questions

The Check Yourself questions at the end of each part are not decoration. If you cannot answer them, reread that part rather than moving on. They are also a ready-made oral quiz for classroom use.

Notes for teaching from this material

- Parts 1 to 3 are one class each. Part 1 needs the full session, since terminology plus the recursion connection is a lot for a beginner.
- Parts 5 and 6 are best taught together, live-coding six functions in a row so the students see the template repeat.
- Part 7 works well as a discussion: ask the class for the brute force first, let them find the repeated work themselves, then show the rewrite. That moment is the whole point of the course.
- Part 12 (Morris) can be skipped for a weaker batch without damaging anything else.
- Part 13 is best taught after students have failed at a few Medium problems. It lands much harder then.
- Part 14 gives four weeks of homework. Part 15 is the handout for the day before an interview or contest.

Conventions used

- Code is C++, in the style LeetCode expects, with NULL rather than nullptr for readability at the whiteboard. Both are fine.
- Heights are counted in **nodes** in code (empty tree gives 0) and in **edges** in prose, and every place where this matters is called out explicitly.
- N is the number of nodes, H is the height, W is the maximum width of a level.
- Problem numbers in brackets refer to LeetCode. Three view problems come from GeeksforGeeks and are marked.